
```

% =====
%                                     MAIN BODY
% =====
clear all;
clearvars -global tspan Y;
clc;

% --- Analysis Control ---
% OPTION 1: Use pitch data directly from the simulation.
% OPTION 2: Use pitch data reconstructed by the ERA method.
analysis_option = 2; % SET YOUR OPTION HERE

% --- Stiffness parameters for the airfoil ---
Sup = [1 1.5 0 2 0 0]; % Supercritical example
gammaA = Sup(1,1); gammaAAA = Sup(1,2); gammaAAAAA = Sup(1,3);
gammaX = Sup(1,4); gammaXXX = Sup(1,5); gammaXXXXX = Sup(1,6);

% --- Initial condition generation ---
Us = 7.45; % Default flow speed
ICM = 2;
fprintf('Constructing initial conditions (ICM=%d)...\\n', ICM);
if ICM == 1
    x0_IC = [60*pi/180; 0; 0; 0; 0; 0; 0; 0];
elseif ICM == 2
    gust_type = 2; W0 = 0.1; T = 10; dtg = 0.05; tmaxg = 2*T;
    tspang = (0:dtg:tmaxg);
    x0g = zeros(8,1); % IC before gust
    [~, xic] = ode45(@(t,x)
aeroelastic_Gust(t,x,W0,T,gust_type,gammaA,gammaAAA,gammaAAAAA,gammaX,gammaXXX
,gammaXXXXX,Us),tspang,x0g);
    x0_IC = xic(end,:);
elseif ICM == 3
    t_IM = (0:0.1:3000)'; UsIM = 8;
    x0IM = [5*pi/180,0,0,0,0,0,0,0];
    [~,xIM] = ode45(@(t,x)
aeroelastic(t,x,gammaA,gammaAAA,gammaAAAAA,gammaX,gammaXXX,gammaXXXXX,UsIM),t_
IM,x0IM);
    x0_IC = (xIM(end,:));
end
fprintf('Initial conditions ready.\\n');

% =====
%                                     GROWTH RATE (mu) ANALYSIS ACROSS FLOW SPEEDS
% =====
usValues = linspace(7.5, 7.6, 3);
dt = 0.1; tmax = 10000;
tspan = 0:dt:tmax;
allGrowthData = cell(1, numel(usValues));

fprintf('\\nAnalyzing growth rate (mu) for different flow speeds...\\n');
if analysis_option == 1
    fprintf('Analysis Mode: Using DIRECT pitch data.\\n');

```

```

elseif analysis_option == 2
    fprintf('Analysis Mode: Using RECONSTRUCTED pitch data.\n');
end

for i = 1:numel(usValues)
    thisUs = usValues(i);
    fprintf('Running for Us = %.3f m/s...\n', thisUs);

    % --- Step 1: Solve the ODE to get the original system response ---
    [tSol, xSol] = ode45(@(t,x) aeroelastic(t, x, ...
        gammaA, gammaAAA, gammaAAAAA, gammaX, gammaXXX, gammaXXXXX,
thisUs), ...
        tspan, x0_IC);

    % --- Step 2: Choose data source based on analysis_option ---
    xSol_for_analysis = xSol; % Default to original data
    if analysis_option == 2
        fprintf('    -> Reconstructing signal with ERA...\n');
        % Define parameters for the ERA reconstruction
        era_params.start_index = 50000;
        era_params.r_era = 5000;
        era_params.s_era = 5000;
        era_params.n_r = 2; % Reconstruct using 2 dominant modes

        % Call the ERA function
        reconstructed_pitch = reconstructSignalWithERA(tSol, xSol, tspan,
era_params);

        % Replace the pitch column in the solution matrix with the
reconstructed data
        xSol_for_analysis(:, 1) = reconstructed_pitch;
    end

    % --- Step 3: Plot response & envelope, and get envelope data ---
    % The function now takes the ODE solution directly
    [tData, rData] = plotResponseAndEnvelope(tSol, xSol_for_analysis, thisUs,
analysis_option, xSol(:,1));

    % Check if the previous function returned valid data
    if isempty(tData) || isempty(rData)
        warning('Cannot generate plot for Us=%.3f due to missing envelope
data.', thisUs);
        allGrowthData{i} = struct('r', [], 'mu', [], 'polyfit', []);
        continue; % Skip to the next speed
    end

    % --- Step 4: Continue with the rest of the analysis ---
    polyCoeffs_mu_r = plotMuVsR(tData, rData, thisUs);
    rdot = diff(rData) ./ diff(tData);
    r_for_analysis = (rData(1:end-1) + rData(2:end)) / 2;
    mu_for_analysis = rdot ./ r_for_analysis;

    allGrowthData{i} = struct('r', r_for_analysis, 'mu', mu_for_analysis,
'polyfit', polyCoeffs_mu_r);

```

```

end

% =====
%                               CRITICAL SPEED CALCULATION FOR GIVEN r
% =====
% Define r range
rList = 0.00 : 0.005 : 0.03;
% We'll store the critical speed for each r
UcList = zeros(size(rList));

fprintf('\nCalculating critical speed (Uc) for each r...\n');
for ir = 1:numel(rList)
    rChosen = rList(ir);

    % 1) Evaluate mu(rChosen) at each U_s using the stored curve fits
    mu_vs_Us = zeros(1, numel(usValues));
    for i = 1:numel(usValues)
        % Check if a polynomial fit exists for this speed
        if ~isempty(allGrowthData{i}.polyfit) &&
~any(isnan(allGrowthData{i}.polyfit))
            % Evaluate the polynomial at the chosen r to get mu
            mu_vs_Us(i) = polyval(allGrowthData{i}.polyfit, rChosen);
        else
            mu_vs_Us(i) = NaN;
        end
    end

    % Remove any NaN values that resulted from failed fits
    valid_idx = ~isnan(mu_vs_Us);
    if sum(valid_idx) < 3
        fprintf('Not enough valid growth rate data to fit for r=%.3f.
Skipping.\n', rChosen);
        UcList(ir) = NaN;
        continue;
    end

    % 2) Fit a 2nd-degree polynomial  $p(U_s) = a_2 U_s^2 + a_1 U_s + a_0$ 
    polyCoeffs = polyfit(usValues(valid_idx), mu_vs_Us(valid_idx), 2);
    a2 = polyCoeffs(1); a1 = polyCoeffs(2); a0 = polyCoeffs(3);

    % 3) Solve the equation  $p(U_s) = 0$  for the critical speed Uc
    discriminant = a1^2 - 4*a2*a0;
    if discriminant < 0
        Uc = NaN; % No real roots
    else
        % Find two roots of the quadratic equation
        U1 = (-a1 + sqrt(discriminant)) / (2*a2);
        U2 = (-a1 - sqrt(discriminant)) / (2*a2);

        % Choose the physically meaningful root
        possibleRoots = [U1, U2];
        positiveRoots = possibleRoots(possibleRoots > 0 & imag(possibleRoots)
== 0);
        if ~isempty(positiveRoots)

```

```

        % Define the target value and find the root closest to it.
        targetValue = 7.6;
        [~, minIndex] = min(abs(positiveRoots - targetValue));
        Uc = positiveRoots(minIndex);
    else
        Uc = NaN;
    end
end
UcList(ir) = Uc;
end

% =====
%                               STEADY-STATE AMPLITUDE CALCULATION
% =====
if exist('steady_state_amplitudes.mat', 'file') == 2
    fprintf('\nFound existing amplitude data file. Loading...\n');
    load('steady_state_amplitudes.mat');
else
    fprintf('\nCalculating steady-state amplitudes...\n');
    usVec = linspace(7.5, 7.7, 100);
    amps_pitch = zeros(size(usVec));
    amps_plunge = zeros(size(usVec));

    for i = 1:length(usVec)
        Us_current = usVec(i);
        [amps_pitch(i), amps_plunge(i)] = amplitudeconver(Us_current,
x0_IC, ...
                                gammaA, gammaAAA, gammaAAAAA, ...
                                gammaX, gammaXXX, gammaXXXXX);
    end
    save('steady_state_amplitudes.mat', 'usVec', 'amps_pitch', 'amps_plunge');
    fprintf('\nSteady-state amplitude data saved to
steady_state_amplitudes.mat\n');
end

% =====
%                               PLOTTING RESULTS
% =====
% --- Combined Plot of mu vs r ---
figure('Name', 'Combined Growth Rate vs. Amplitude', 'Color', 'w');
hold on; grid on;
colors = lines(numel(usValues)); % Generate distinct colors for each Us
for i = 1:numel(usValues)
    if ~isempty(allGrowthData{i}.r) && ~any(isnan(allGrowthData{i}.polyfit))
        thisUs = usValues(i);

        % Plot the scattered data points
        plot(allGrowthData{i}.r, allGrowthData{i}.mu, 'o', 'MarkerSize',
6, ...
            'Color', colors(i,:), 'MarkerFaceColor', colors(i,:), ...
            'DisplayName', sprintf('$U_s = %.3f$ m/s', thisUs));

        % Plot the corresponding fitted curve
        r_fit_line = linspace(0, 0.07, 200);

```

```

        mu_fit_line = polyval(allGrowthData{i}.polyfit, r_fit_line);
        plot(r_fit_line, mu_fit_line, '-', 'LineWidth', 2, 'Color',
colors(i,:), ...
        'HandleVisibility', 'off'); % Hide from legend
    end
end
title('Growth Rate vs. Amplitude for Different Flow Speeds');
xlabel('Amplitude, $r$ [rad]', 'Interpreter', 'latex');
ylabel('Growth Rate, $\mu = \dot{r}/r$ [1/s]', 'Interpreter', 'latex');
legend('show', 'Location', 'best', 'Interpreter', 'latex');
ax = gca; ax.FontSize = 12;
hold off;

% --- Bifurcation Diagram for Pitch ---
figure('Color', 'w', 'Name', 'Bifurcation Diagram: Pitch');
hold on; grid on;
plot(UcList, rList, 'bo-', 'LineWidth', 2, 'MarkerSize', 6, 'DisplayName',
'Unstable Limit Cycle ($\mu=0$)');
plot(usVec, amps_pitch, 'k-', 'LineWidth', 2.5, 'DisplayName', 'Stable Limit
Cycle (Converged Amp.)');
xlabel('Flow Speed ($U_s$) [m/s]'); ylabel('Pitch Amplitude [rad]');
title('Bifurcation Diagram for Pitch Response');
legend('show', 'Location', 'best', 'Interpreter', 'latex');
ax = gca; ax.FontSize = 12;
hold off;

```

Constructing initial conditions (ICM=2)...

Initial conditions ready.

Analyzing growth rate (μ) for different flow speeds...

Analysis Mode: Using RECONSTRUCTED pitch data.

Running for $U_s = 7.500$ m/s...

-> Reconstructing signal with ERA...

```

=====
MODAL ANALYSIS RESULTS
=====

```

Mode 1:

Natural Frequency (ω_n): 0.0911 rad/s (0.0145 Hz)

Damped Frequency (ω_d): 0.0911 rad/s (0.0145 Hz)

Damping Ratio (ζ): 0.020620 [STABLE]

Mode Shape:

State 1: 7.071068e-01

State 2: -1.959999e-02

```

=====
Total unique modes displayed: 1
=====

```

Running for $U_s = 7.550$ m/s...

-> Reconstructing signal with ERA...

```

=====
MODAL ANALYSIS RESULTS

```

```
=====
Mode 1:
  Natural Frequency ( $\omega_n$ ): 0.0895 rad/s (0.0142 Hz)
  Damped Frequency ( $\omega_d$ ): 0.0895 rad/s (0.0142 Hz)
  Damping Ratio ( $\zeta$ ): 0.012046 [STABLE]
  Mode Shape:
    State 1: 7.071068e-01
    State 2: -7.554752e-04
=====
```

```
Total unique modes displayed: 1
=====
```

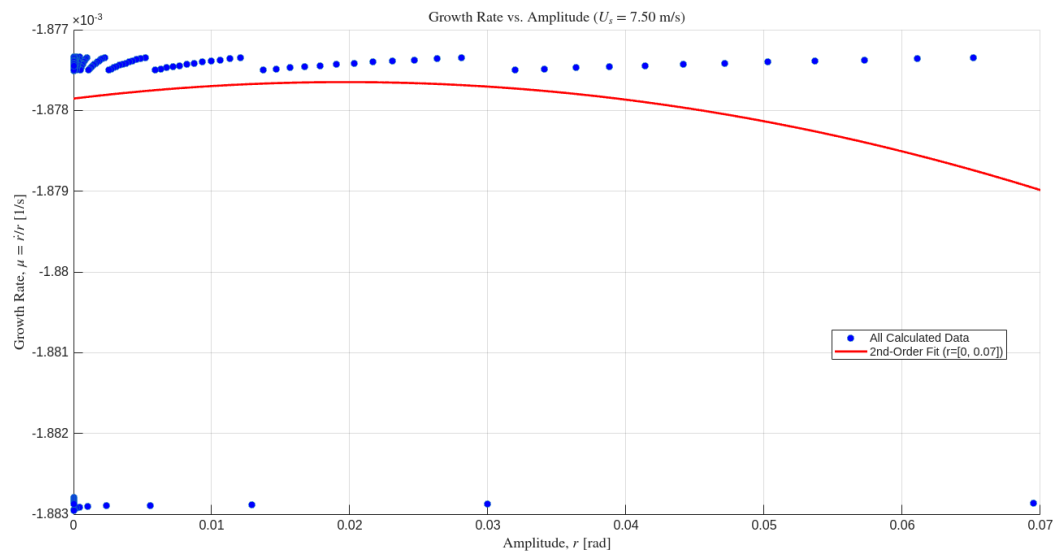
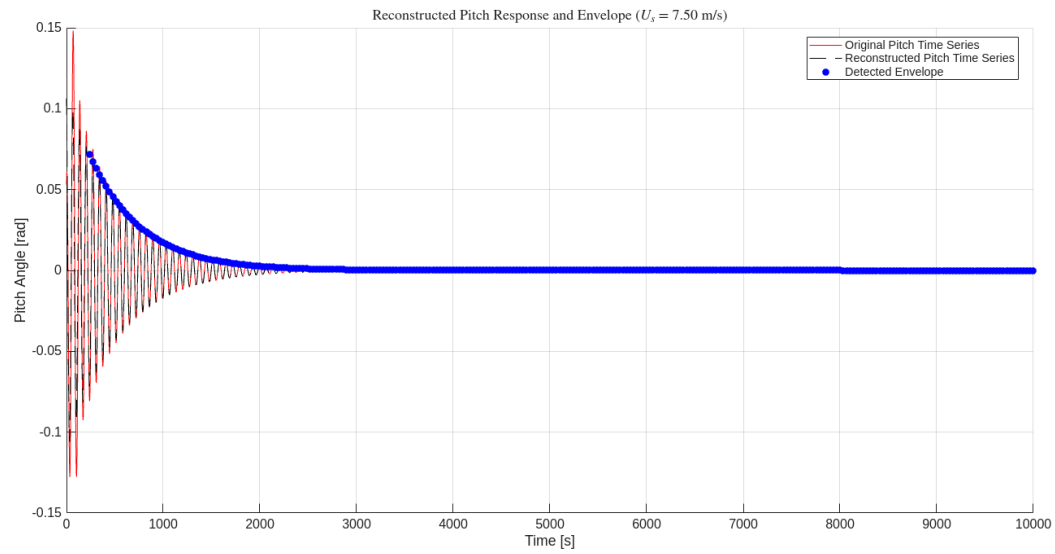
```
Running for  $U_s = 7.600$  m/s...
-> Reconstructing signal with ERA...
```

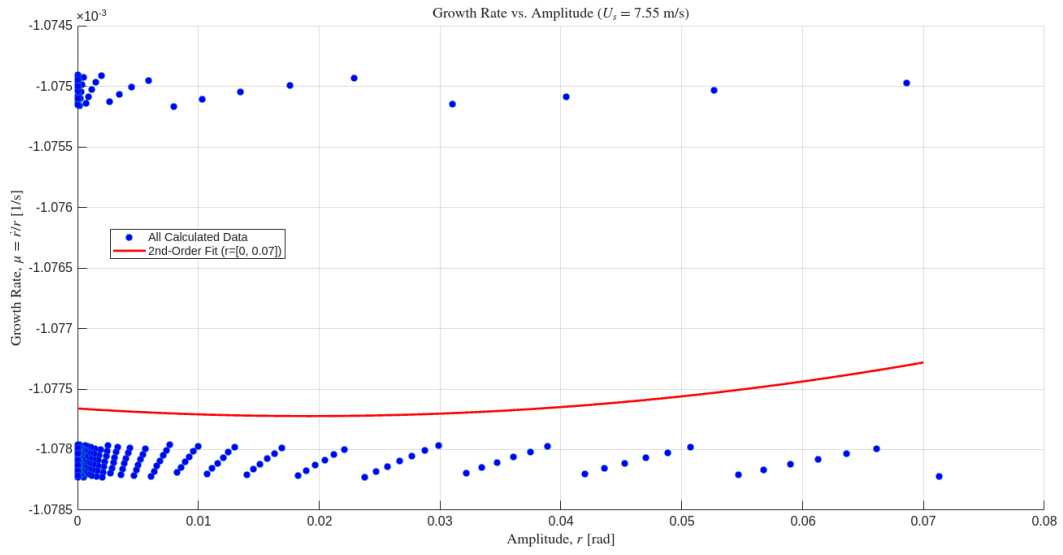
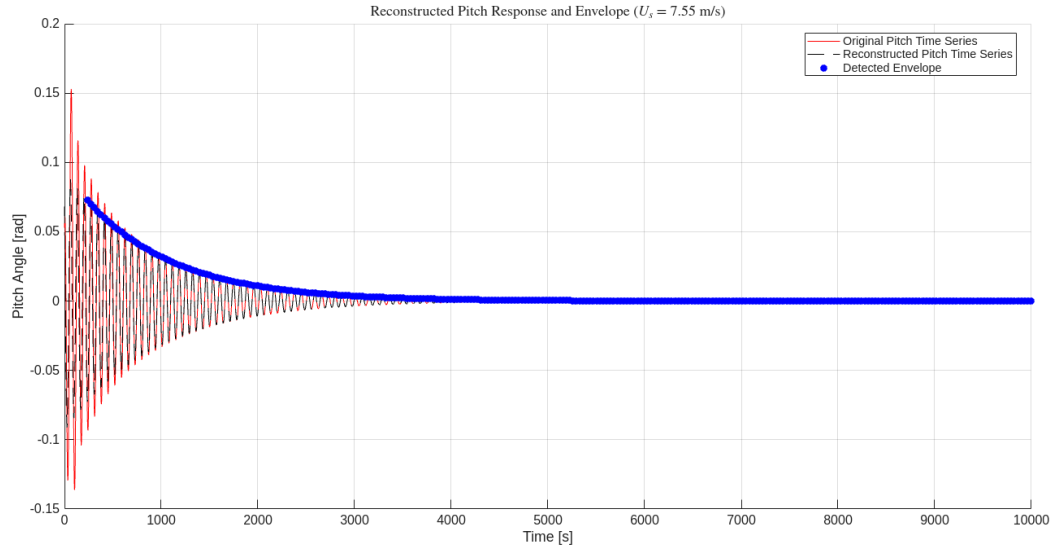
```
=====
MODAL ANALYSIS RESULTS
=====
```

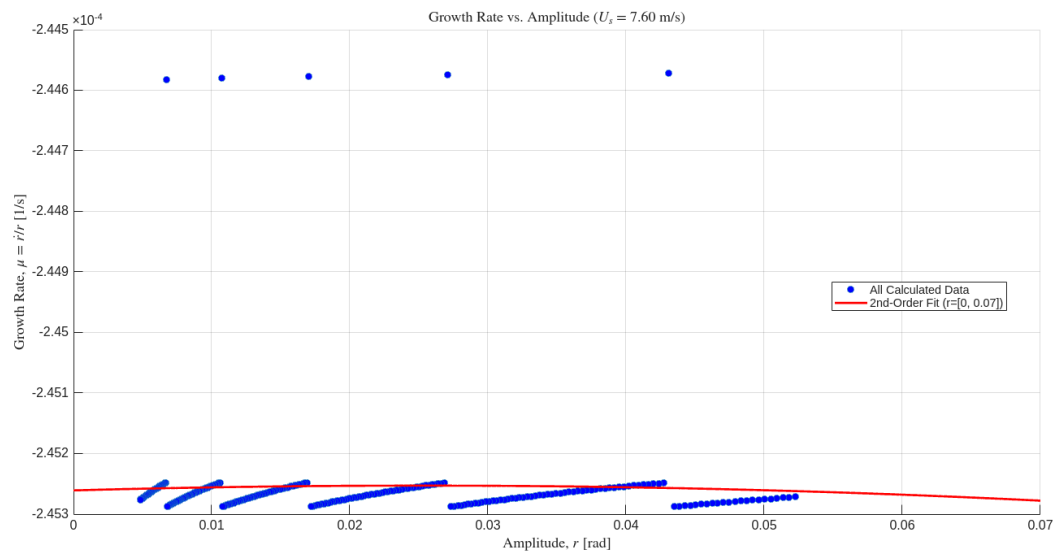
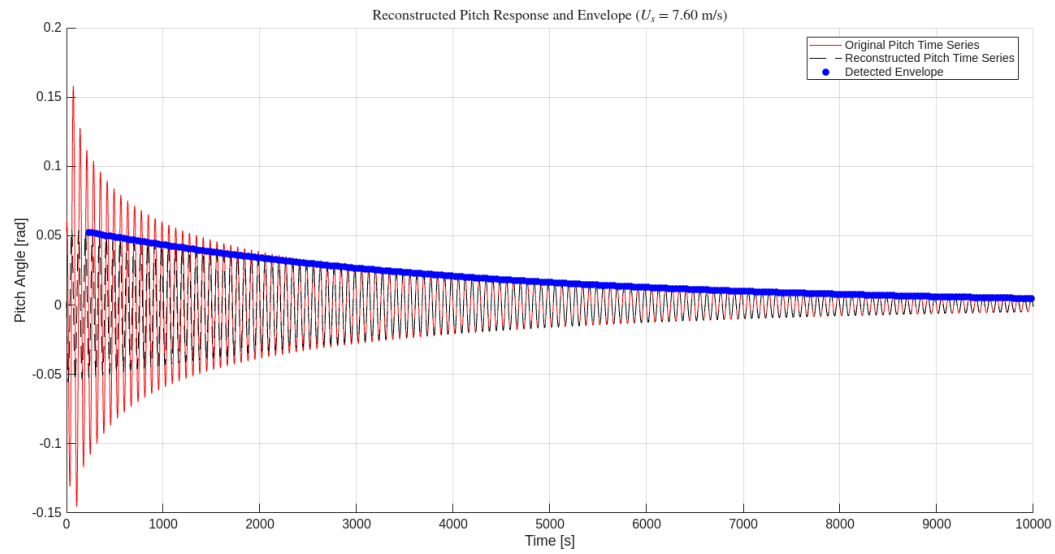
```
Mode 1:
  Natural Frequency ( $\omega_n$ ): 0.0880 rad/s (0.0140 Hz)
  Damped Frequency ( $\omega_d$ ): 0.0880 rad/s (0.0140 Hz)
  Damping Ratio ( $\zeta$ ): 0.002787 [STABLE]
  Mode Shape:
    State 1: 7.071068e-01
    State 2: 5.413625e-05
=====
```

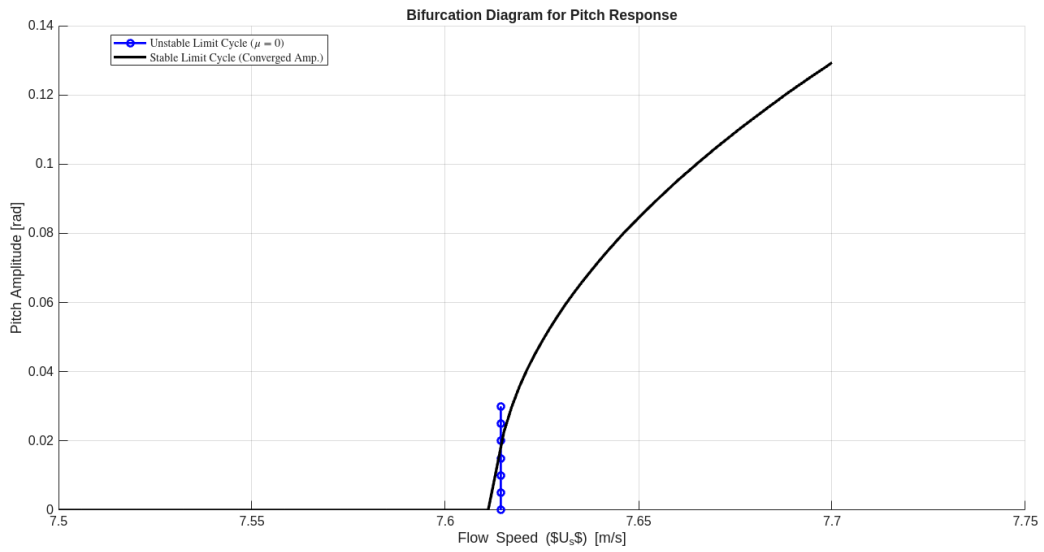
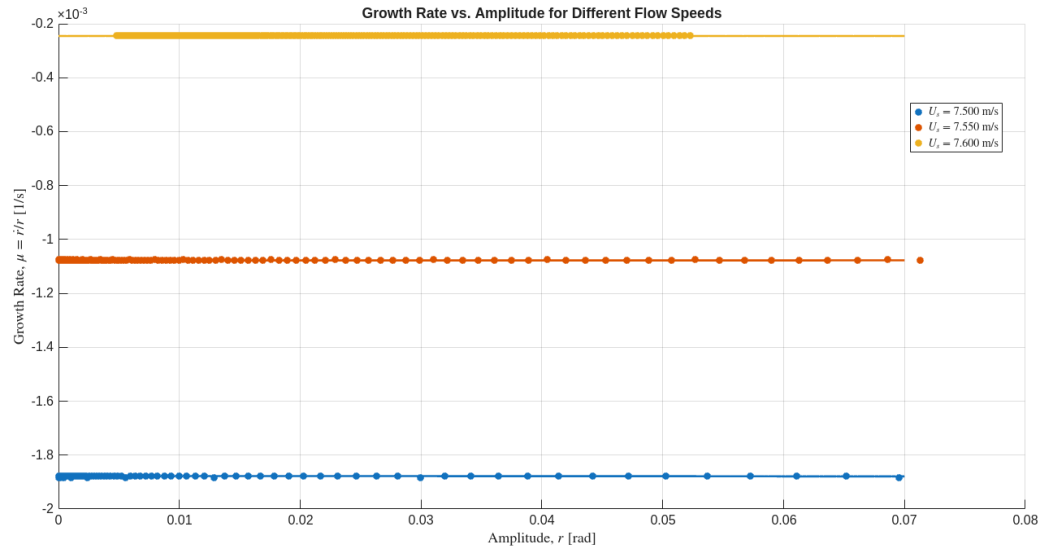
```
Total unique modes displayed: 1
=====
```

```
Calculating critical speed ( $U_c$ ) for each  $r$ ...
Found existing amplitude data file. Loading...
```









Published with MATLAB® R2025b